Northeastern University

EECE 7205
Fundamentals of Computer Engineering

Greedy Algorithms with Arrays:
EV Charging Station

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201

November 2026

Contents

The Problem **2**

Task 1: Understanding the Problem **2**

Task 2: Greedy Idea **3**

Task 3: Pseudocode **3**

Task 4: Hand Trace **3**

Task 5: Time Complexity **4**
- Frequency Count 4
- Final $T(n)$ 5
- Space Complexity 6
- Comparison with Checking Every Pair 6

Task 6: C++ Program **6**
- Vector Version 6
- Array Version (as required) 8
- How the Heap Sort Works 9
- Program Output 9
- Test Results 10

Task 7: Think Further **10**

The Problem

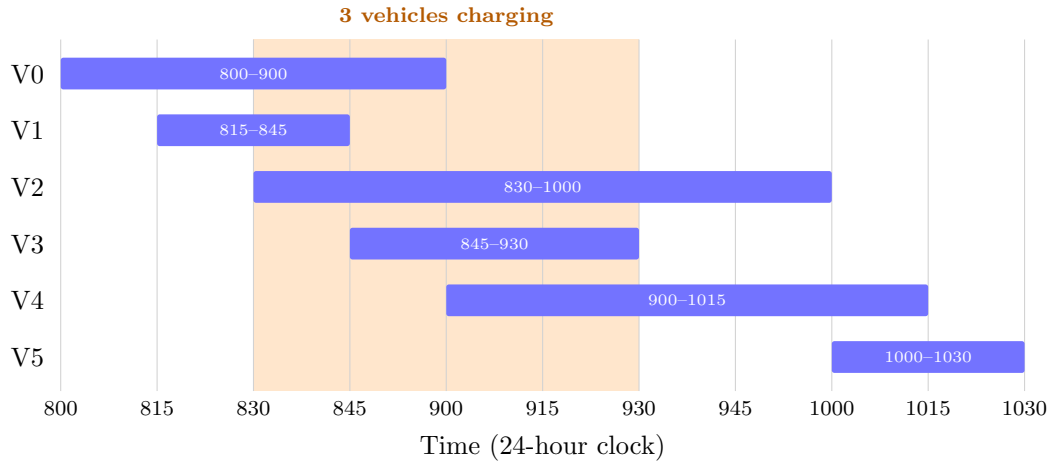
A campus EV charging station records each vehicle's arrival and departure time as a 24-hour integer (e.g. `830` means 8:30). Each vehicle needs its own charger from arrival until departure. We must find the smallest number of chargers so that no vehicle waits. A vehicle leaving at time t frees its charger at t, so a vehicle arriving at exactly t may use it.

Vehicle	0	1	2	3	4	5
`arrive`	800	815	830	845	900	1000
`depart`	900	845	1000	930	1015	1030

Expected output: `Chargers needed: 3`

Task 1: Understanding the Problem

Each vehicle is drawn as a bar from its arrival to its departure. The shaded band marks the period when the most vehicles are charging.



Counting the bars in each interval between consecutive events:

Interval	Vehicles charging	Change at start of interval	In use
800–815	V0	V0 arrives	1
815–830	V0, V1	V1 arrives	2
830–845	V0, V1, V2	V2 arrives	**3**
845–900	V0, V2, V3	V1 leaves, V3 arrives	**3**
900–930	V2, V3, V4	V0 leaves, V4 arrives	**3**
930–1000	V2, V4	V3 leaves	2
1000–1015	V4, V5	V2 leaves, V5 arrives	2
1015–1030	V5	V4 leaves	1

Answer: The most vehicles charging at the same time is **3**. This first happens at **8:30** (when vehicle 2 arrives, with vehicles 0 and 1 still charging) and the count stays at 3 from 830 until 930. At 845 and 900 one vehicle leaves at exactly the moment another arrives, so because of the equal-times rule the freed charger is reused and the count does not rise to 4. Therefore the station needs **3 chargers**.

Task 2: Greedy Idea

Sort the arrival times and the departure times separately, then sweep through all events in time order, always taking the earlier of the next arrival and the next departure (and taking the departure first when they are equal, so a freed charger can be reused immediately). An arrival *increases* the number of chargers in use by one, and a departure *decreases* it by one. The number in use at any moment equals the number of vehicles physically present, so the station needs at least the largest such value; and that many is also enough, because whenever a vehicle arrives the greedy choice is to hand it any charger that has been freed, so a new charger is only ever needed when the count rises above its previous maximum.

Task 3: Pseudocode

Algorithm 1 MinChargers($arrive[0..n-1]$, $depart[0..n-1]$, n)

```
1:  sort arrive in ascending order
2:  sort depart in ascending order
3:  i ← 0                                              ▷ index of next arrival
4:  j ← 0                                              ▷ index of next departure
5:  inUse ← 0,    most ← 0
6:  while i < n do
7:      if j < n and arrive[i] ≥ depart[j] then        ▷ departure first on a tie
8:          inUse ← inUse − 1                          ▷ charger freed
9:          j ← j + 1
10:     else
11:         inUse ← inUse + 1                          ▷ charger taken
12:         if inUse > most then
13:             most ← inUse
14:         i ← i + 1
15: return most
```

Handling equal times. The test uses $\geq$, not $>$. When $arrive[i] = depart[j]$ the departure is processed first, so the charger is released before the arriving vehicle takes one. Using $>$ instead would count both vehicles at once and overestimate the answer (it would give 4 on the sample data).

Why the loop can stop when $i = n$. After the last arrival only departures remain, and departures can only lower *inUse*, so *most* cannot change.

Why the check $j < n$. It stops the loop from reading past the end of *depart*. With valid data (every vehicle leaves after it arrives) j always stays below i (see Task 7), so the check never actually fails. For this reason both C++ programs in Task 6 leave it out and test only `arr[i] >= dept[j]`; this gives the same result on every valid input.

Task 4: Hand Trace

Sorted arrive: `800  815  830  845  900  1000`

Sorted depart: `845  900  930  1000  1015  1030`

"Next arrival" is $arrive[i]$ and "Next departure" is $depart[j]$ *before* the step is executed.

Step	Next arrival	Next departure	Event	Chargers in use	Most so far
1	800	845	Arrival ($800 < 845$)	1	1
2	815	845	Arrival ($815 < 845$)	2	2
3	830	845	Arrival ($830 < 845$)	3	3
4	845	845	Departure (tie, $845 \geq 845$)	2	3
5	845	900	Arrival ($845 < 900$)	3	3
6	900	900	Departure (tie, $900 \geq 900$)	2	3
7	900	930	Arrival ($900 < 930$)	3	3
8	1000	930	Departure ($1000 \geq 930$)	2	3
9	1000	1000	Departure (tie, $1000 \geq 1000$)	1	3
10	1000	1015	Arrival ($1000 < 1015$)	2	3
11	— ($i = n$)	1015	Departure (loop has ended)	1	3
12	— ($i = n$)	1030	Departure (loop has ended)	0	3

The algorithm exits after step 10 because all six arrivals have been processed. Steps 11 and 12 (in grey) show the two remaining departures for completeness; they only lower the count, so the maximum stays at 3.

Result: `Chargers needed: 3`, which matches the expected output and the peak found in Task 1.

Task 5: Time Complexity

Frequency Count

Let n be the number of vehicles. The table counts the vector version from Task 6; the array version differs only in lines 1–2 (see below). Each statement is given a constant cost c_k, and its frequency is the number of times it runs in the worst case. The sweep loop moves either i or j forward by one each pass, so it walks over both sorted arrays of size n.

#	Statement	Cost	Frequency	Total
1	`arr[i] = vehicles[i].first;` (copy loop)	c_1	n	$c_1 n$
2	`dept[i] = vehicles[i].second;` (copy loop)	c_2	n	$c_2 n$
3	`heapSort(arr, n);`	c_3	$n \log n$	$c_3\, n \log n$
4	`heapSort(dept, n);`	c_4	$n \log n$	$c_4\, n \log n$
5	`i=0; j=0; chargers=0; maxChargers=0;`	c_5	1	c_5
6	`while (i < n)`	c_6	$2n$	$2c_6 n$
7	`if (arr[i] >= dept[j])`	c_7	$2n-1$	$c_7(2n-1)$
8	`chargers--; j++;`	c_8	$n-1$	$c_8(n-1)$
9	`chargers++;`	c_9	n	$c_9 n$
10	`maxChargers = max(maxChargers, chargers);`	c_{10}	n	$c_{10} n$
11	`i++;`	c_{11}	n	$c_{11} n$
12	`return maxChargers;`	c_{12}	1	c_{12}

Where the frequencies come from.

- Lines 1–2 copy the n pairs into the two arrays `arr` and `dept`: $n + n = 2n$ steps. Copying the `vehicles` parameter when the function is called (it is passed by value) is another $O(n)$ step and does not change the Big-O result. In the array version lines 1–2 are `cin >> arr[i];` and `cin >> dept[i];` instead, which also run n times each, so every frequency, and therefore $T(n)$, is the same for both versions.
- Lines 3–4 heap sort each array in place. Building the heap is $O(n)$ and the $n-1$ extractions each sift down at most $\log n$ levels, so each sort is about $n \log n$ steps: $2n \log n$ in total.
- Lines 9–11 (an arrival) run exactly n times, once per vehicle. Line 10 runs on every arrival, so it is also counted n times.
- Line 8 (a departure) runs at most $n-1$ times, because the loop ends as soon as the last arrival is handled.
- So the loop body runs at most $n + (n-1) = 2n-1$ times (line 7), and the loop test runs one extra time when it fails, $2n$ times (line 6). This is the pass over the two arrays of size n.

Final $T(n)$

Adding the Total column:

$$T(n) = (c_3 + c_4)\, n \log n + (c_1 + c_2 + 2c_6 + 2c_7 + c_8 + c_9 + c_{10} + c_{11})\, n + (c_5 + c_{12} - c_7 - c_8).$$

With every $c_k = 1$:

$$T(n) = 2n \log_2 n + 10n.$$

Grouping by part of the algorithm:

$$T(n) = \underbrace{O(n \log n)}_{\text{two sorts}} + \underbrace{O(2n)}_{\text{pass over 2 arrays of size } n}.$$

Big-O drops constant factors, so $O(2n) = O(n)$, and since $n \log n$ grows faster than n, the sorting term dominates:

$$\boxed{T(n) = O(n \log n) + O(2n) = O(n \log n)}$$

Space Complexity

The function `findminchargers` takes `vehicles` by value, with no `&` in the parameter:

```
const vector<pair<int,int>> vehicles
```

so the call makes a copy of the n pairs, which is $2n$ integers. It then stores two more arrays of n integers (`arr` and `dept`) plus four scalar variables (`i`, `j`, `chargers`, `maxChargers`):

$$S(n) = 2n + n + n + 4 = 4n + 4 \quad \Longrightarrow \quad \boxed{S(n) = O(4n) = O(n)}$$

The array version reads the times straight into `arr` and `dept` (declared with room for 100 values, the largest n allowed) and passes them to `findminchargers` without copying, so it uses $S(n) = n + n + 4 = 2n + 4$. Both are $O(n)$.

Heap sort works in place and needs only $O(1)$ extra memory (a few index variables), so it does not change the result.

Comparison with Checking Every Pair

The simple method takes each vehicle k and compares it with every other vehicle to count how many are charging at the moment $arrive[k]$ (the peak always begins at some arrival). That is n checks for each of n vehicles, so it costs $\boldsymbol{O(n^2)}$. For $n = 100$ this is about $100^2 = 10{,}000$ comparisons, while the greedy method needs roughly $T(100) = 2 \cdot 100 \log_2 100 + 10 \cdot 100 \approx 2{,}300$ steps. The gap grows quickly with n: for $n = 10{,}000$ it is about 10^8 versus $T(10{,}000) \approx 3.7 \times 10^5$.

Task 6: C++ Program

The full source code is also available on GitHub:

`https://github.com/roh9031/EECE---7205-Class-Assignment-/tree/main/Class_Assignment_6_Bidnurmath`

Two versions of the program are included. Both follow the pseudocode from Task 3 and use the same heap sort and greedy sweep (`chargers` plays the role of *inUse* and `maxChargers` the role of *most*). The first version was written with `vector`. Since the assignment asks for **plain arrays (no `vector`)** and input read with `cin`, a second version uses plain arrays and reads the input in the required format. The array version is the one that meets the assignment requirements; both give the same answers.

Vector Version

The arrival and departure times are passed in as {`arrive, depart`} pairs, which are split into two arrays, `arr` and `dept`. Each array is sorted with a hand-written **heap sort**, and then the greedy sweep counts the chargers in use. The vehicle list is passed by value as a `const` parameter, so the function works on its own copy and cannot change the caller's data. The three test cases are written directly in `main`.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
void siftDown(vector<int>& arr, int root, int size) {
    while (2 * root + 1 < size) {
        int child = 2 * root + 1;
        if (child + 1 < size && arr[child + 1] > arr[child]) {
            child = child + 1;
        }
        if (arr[root] >= arr[child]) {
            break;
        }
        swap(arr[root], arr[child]);
        root = child;
    }
}
void heapSort(vector<int>& arr, int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        siftDown(arr, i, n);
    }
    for (int end = n - 1; end >= 1; end--) {
        swap(arr[0], arr[end]);
        siftDown(arr, 0, end);
    }
}
// i am assuming the arrival and dept times come in vector int pairs rather than 2 separate vectors named arr and dept
int findminchargers(const vector<pair<int,int>> vehicles) {
    int n = vehicles.size();
    vector<int> arr(n), dept(n);
    for(int i = 0; i < n; i++){
        arr[i] = vehicles[i].first;
        dept[i] = vehicles[i].second;
    }
    heapSort(arr, n);
    heapSort(dept, n);
    int i = 0, j = 0;
    int chargers = 0;
    int maxChargers = 0;
    while(i < n){
        if(arr[i] >= dept[j]){
            chargers--;
            j++;
        } else {
            chargers++;
            maxChargers = max(maxChargers, chargers);
            i++;
        }
    }
    return maxChargers;
}
int main() {
```

```
    cout<<"Test case: all arrive together - {900,1000},{900,1000},{900,1000},{900,1000}\n
    ";
    vector<pair<int,int>> test1 = {{900,1000},{900,1000},{900,1000},{900,1000}};
    cout << "Chargers needed: " << findminchargers(test1)<<"\n" << endl;
    cout<<"Test case: all arrive back to back - {900,1000},{1000,1100},{1100,1200}\n";
    vector<pair<int,int>> test2 = {{900,1000},{1000,1100},{1100,1200}};
    cout << "Chargers needed: " << findminchargers(test2)<<"\n" << endl;
    cout<<"Test case: random timings -
    {800,900},{815,845},{830,1000},{845,930},{900,1015},{1000,1030}\n";
    vector<pair<int,int>> test3 =
    {{800,900},{815,845},{830,1000},{845,930},{900,1015},{1000,1030}};
    cout << "Chargers needed: " << findminchargers(test3) << endl;
    return 0;
}
```

Array Version (as required)

The same algorithm with plain `int` arrays, used because the assignment asks for no `vector`. `main` reads the input with `cin` in the required format: the number of vehicles n (at most 100), then the n arrival times, then the n departure times. These go straight into `arr` and `dept`, which are passed to `findminchargers` and sorted in place, so no copy is made.

```
#include <iostream>
#include <algorithm>
using namespace std;
void siftDown(int arr[], int root, int size) {
    while (2 * root + 1 < size) {
        int child = 2 * root + 1;
        if (child + 1 < size && arr[child + 1] > arr[child]) {
            child = child + 1;
        }
        if (arr[root] >= arr[child]) {
            break;
        }
        swap(arr[root], arr[child]);
        root = child;
    }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        siftDown(arr, i, n);
    }
    for (int end = n - 1; end >= 1; end--) {
        swap(arr[0], arr[end]);
        siftDown(arr, 0, end);
    }
}
// array version the assignment asks for plain arrays (no vector) and input read with cin
// input format n then the n arrival times then the n departure times
int findminchargers(int arr[], int dept[], int n) {
    heapSort(arr, n);
    heapSort(dept, n);
    int i = 0, j = 0;
```

```
    int chargers = 0;
    int maxChargers = 0;
    while(i < n){
        if(arr[i] >= dept[j]){
            chargers--;
            j++;
        } else {
            chargers++;
            maxChargers = max(maxChargers, chargers);
            i++;
        }
    }
    return maxChargers;
}
int main() {
    int n;
    int arr[100], dept[100];   // n is at most 100
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> arr[i];
    }
    for(int i = 0; i < n; i++){
        cin >> dept[i];
    }
    cout << "Chargers needed: " << findminchargers(arr, dept, n) << endl;
    return 0;
}
```

How the Heap Sort Works

`siftDown` moves a value down a max-heap: it compares the value with its larger child and swaps them until the value is at least as large as both children. `heapSort` first builds a max-heap by sifting down every non-leaf node, from index $n/2 - 1$ back to 0. It then repeatedly swaps the largest value (the root) to the end of the array and sifts the new root down over the remaining, shorter heap. Building the heap costs $O(n)$ and the $n - 1$ extractions cost $O(\log n)$ each, so heap sort runs in $O(n \log n)$ time and sorts the array in place with $O(1)$ extra memory.

Program Output

Vector version (one run, tests written in `main`):

```
Test case: all arrive together - {900,1000},{900,1000},{900,1000},{900,1000}
Chargers needed: 4

Test case: all arrive back to back - {900,1000},{1000,1100},{1100,1200}
Chargers needed: 1

Test case: random timings -
    {800,900},{815,845},{830,1000},{845,930},{900,1015},{1000,1030}
Chargers needed: 3
```

Array version (one run per test; the first three lines are the input typed in, the last line is the output):

Sample (vehicle data from page 1):

```
6
800 815 830 845 900 1000
900 845 1000 930 1015 1030
Chargers needed: 3
```

All arrive together:

```
4
900 900 900 900
1000 1000 1000 1000
Chargers needed: 4
```

Back-to-back:

```
3
900 1000 1100
1000 1100 1200
Chargers needed: 1
```

Test Results

Inputs are shown in the `cin` format used by the array version: n, then the arrivals, then the departures. The vector version was run on the same data written as {`arrive, depart`} pairs.

Test	Input	Expected output	Array version	Vector version	Pass?

Sample	`6`
	`800 815 830 845 900 1000`
	`900 845 1000 930 1015 1030`
		`Chargers needed: 3`	`Chargers needed: 3`	`Chargers needed: 3`	**Yes**

All arrive together	`4`
	`900 900 900 900`
	`1000 1000 1000 1000`
		`Chargers needed: 4`	`Chargers needed: 4`	`Chargers needed: 4`	**Yes**

Back-to-back	`3`
	`900 1000 1100`
	`1000 1100 1200`
		`Chargers needed: 1`	`Chargers needed: 1`	`Chargers needed: 1`	**Yes**

The back-to-back case confirms the equal-times rule: each vehicle leaves at the exact moment the next one arrives, so a single charger is reused all morning.

Task 7: Think Further

After sorting, $arrive[k]$ and $depart[k]$ usually belong to different vehicles, but the algorithm never needs to know which departure goes with which arrival. At any moment t, the number of chargers in use is simply

$$(\text{number of arrivals up to } t) \;-\; (\text{number of departures up to } t),$$

with departures at exactly t counted first. Both counts depend only on the *set* of arrival times and the *set* of departure times, not on how they are paired. Sorting each array keeps every time value and only changes the order, so these counts, and therefore their maximum, are unchanged.

In other words, a charger is not tied to a particular vehicle: when any vehicle leaves, its charger becomes available to whoever arrives next. The sweep tracks only how many chargers are busy, which is exactly what the question asks.

The count also never goes negative. Since every vehicle departs after it arrives, the k-th smallest departure time is always later than the k-th smallest arrival time. So the algorithm can never process more departures than arrivals, and $inUse$ always describes a real situation.